



#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

China 2026

Turning Fragmented GPU Clusters Into One Elastic Compute Pool

Chongkang Tan, Ant Group



About Me



Chongkang Tan is a Staff Engineer at Ant Group, where he leads application compute architecture for Infrastructure Compute Services.

His work focuses on efficient GPU, sandbox, and container compute services for agentic applications.

He has spoken at KubeCon, QCon, BDTC, and other technical conferences.

Table of Contents

- Typical Complaints in AI Infrastructure
- Elastic Compute Pool: Abstraction and Architecture
- Cluster Federation, Scheduling, Data and Compute Descriptions
- Lessons from Production
- Future Roadmap



User Complaints in AI Infrastructure

1

Resource Complaints

You have so many GPUs. Why can't I get the few GPUs I requested?

2

Utilization Complaints

Why is utilization still low when demand is so high?

3

Complexity Complaints

Why do I need to understand topology, data locality, runtime versions, and GPU types just to run my job?

Why Is It So Complex to Use AI Compute Effectively?



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

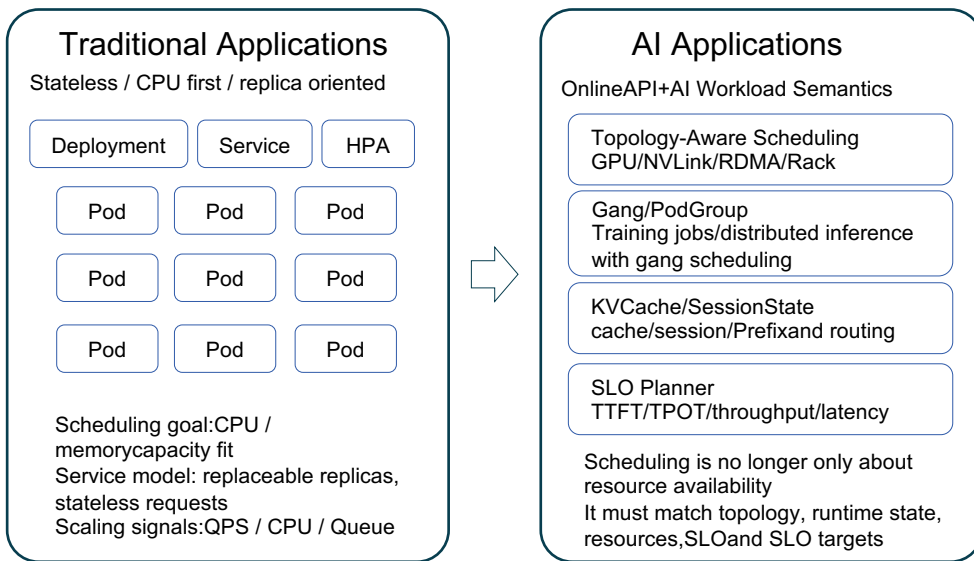
AI Applications Are Different

A Typical Case:

An AI job needs 128 GPUs, means

- GPU type
- topology/scheduling features
- image/model/data availability,
- and a predictable queue time.

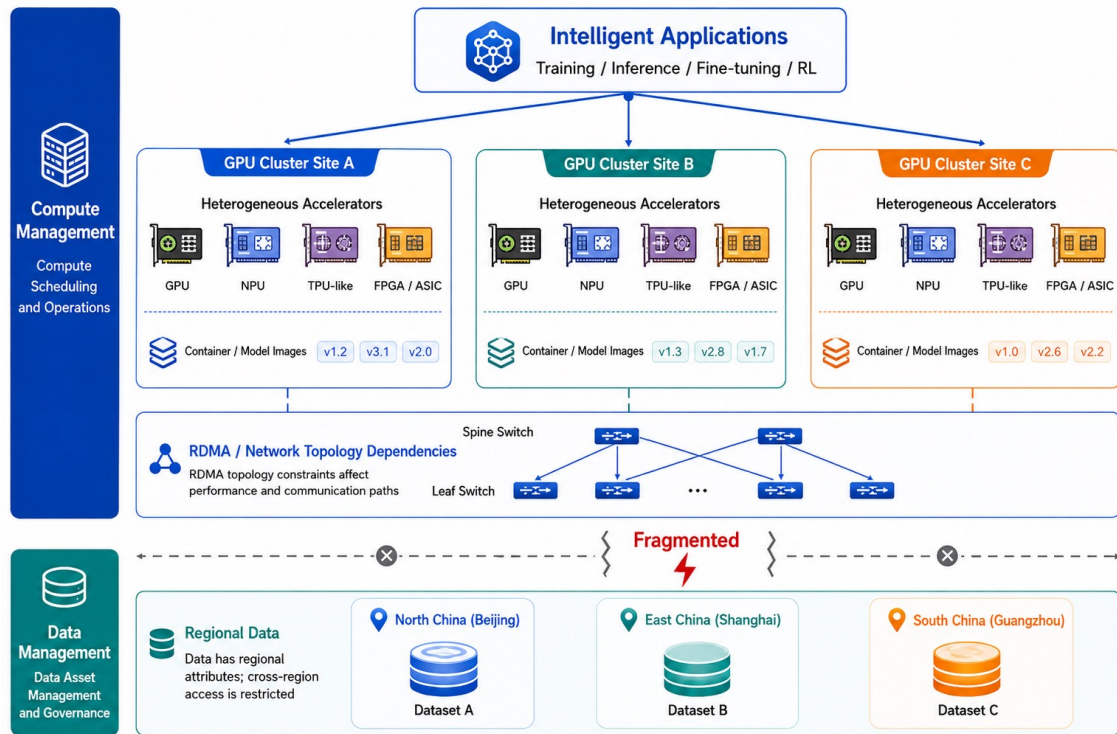
The platform may have thousands of GPUs, but only a small subset is actually usable.



GPU type + topology + scheduling feature + data + SLA

A few extra conditions change the problem complexity grows non-linearly.
Every placement changes system state.

GPU Count is Not Compute Capacity



Challenge:

- Severe compute heterogeneity
- Fragmented IDCs/Resource Pools and Compute Software Stacks
- Strict & unnecessary topology requirements
- Compute and data management are disconnected

Result:

- Low Resource Fulfillment Rate, Low utilization and High complexity

We Need a Better Compute Abstraction



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Elastic Compute Pool: Abstraction and Architecture

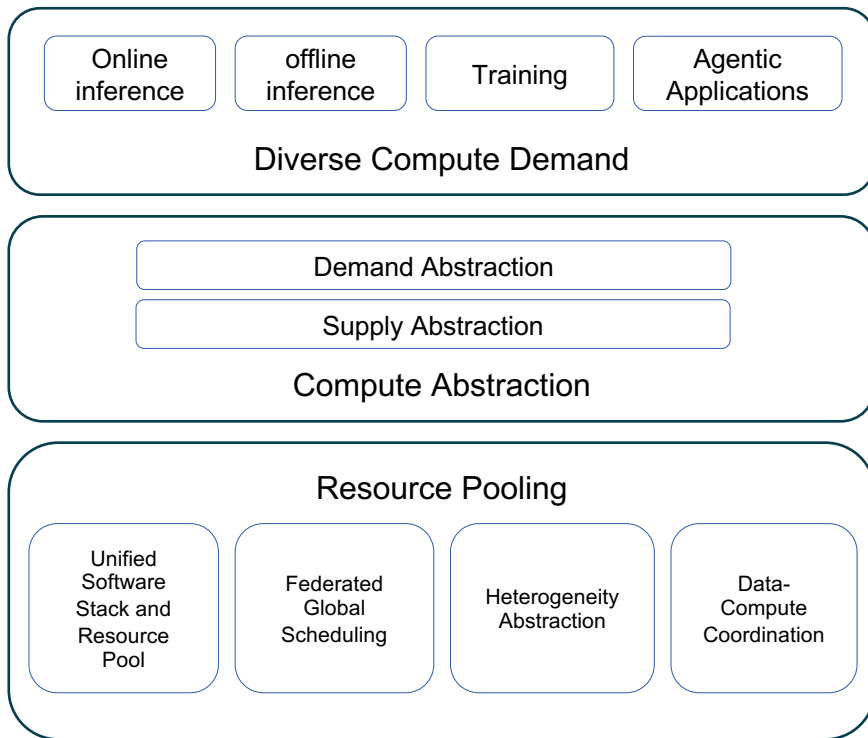


Elastic Compute Pool

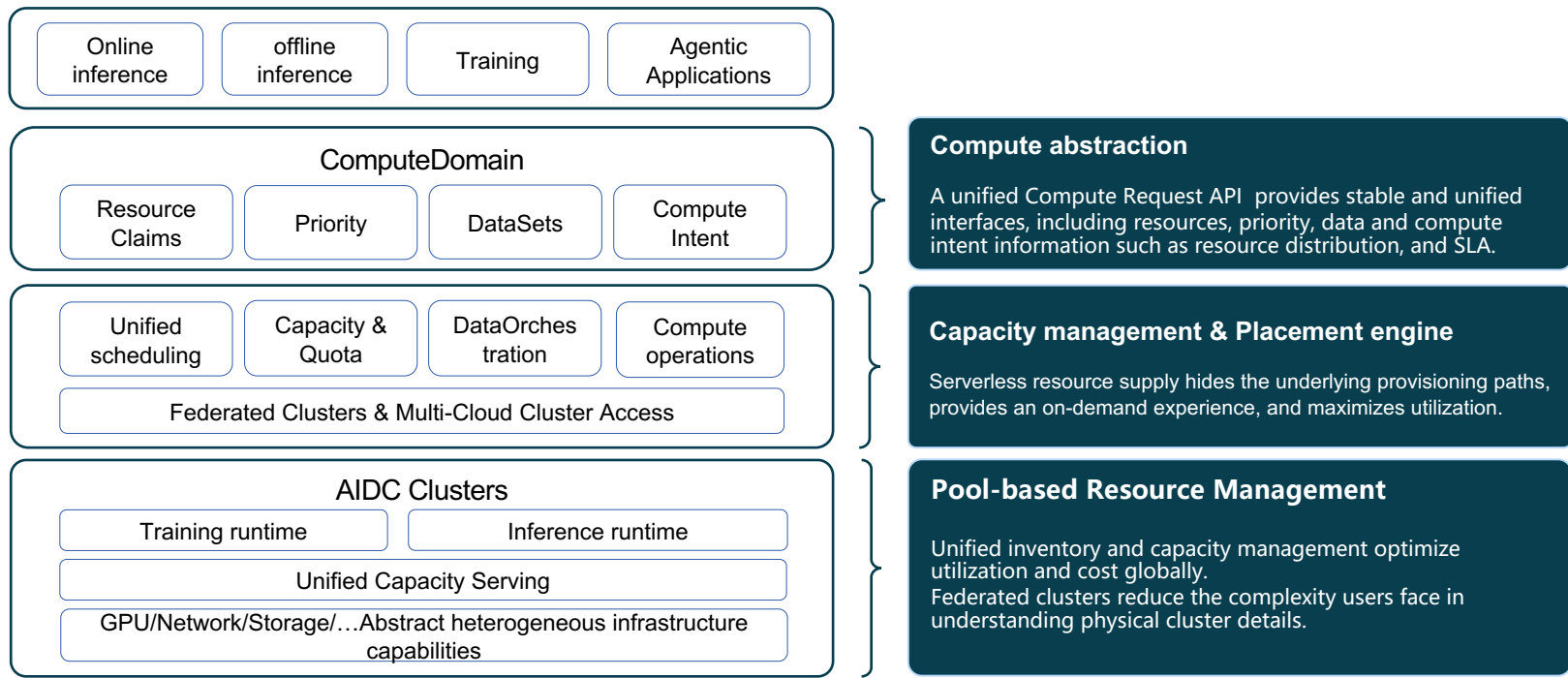
AI workloads need usable compute capacity, not isolated GPU devices.

Elastic Compute Pool

- Turns fragmented heterogeneous clusters into schedulable, policy-driven, data-aware compute capacity.
- Unify the abstraction of resource supply and demand to provide on-demand provisioning and pooled resource operations, meeting the resource requirements of diverse applications.



Abstraction and Architecture



Cluster Federation, Scheduling, Data and Compute Descriptions



KubeCon



CloudNativeCon

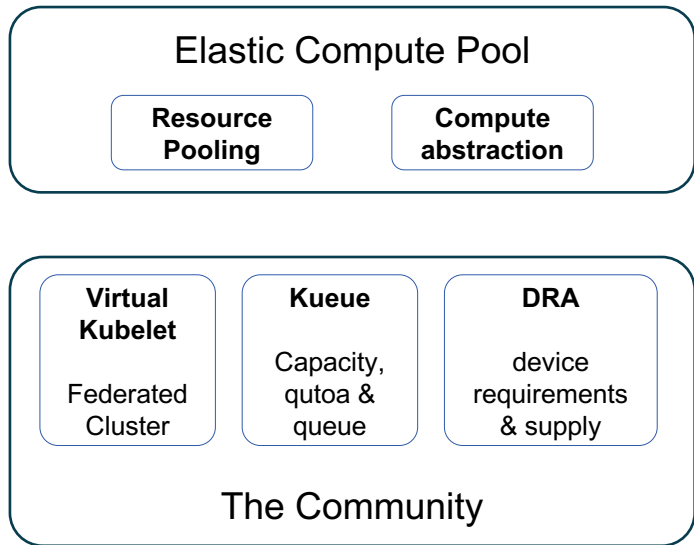


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Built on Kubernetes, Extended for Elastic Compute Pool



We do not need to build everything from scratch, the community gives us the foundation.

- Virtual Kubelet: expose external and federated capacity as Kubernetes nodes
- Kueue: provide Kubernetes-native queueing, quota, and admission control
- DRA: describe device requirements beyond simple resource counts

Our work is to adapt and extend these capabilities for large-scale, production-oriented AI compute service.

Federated Clusters - VirtualKubelet

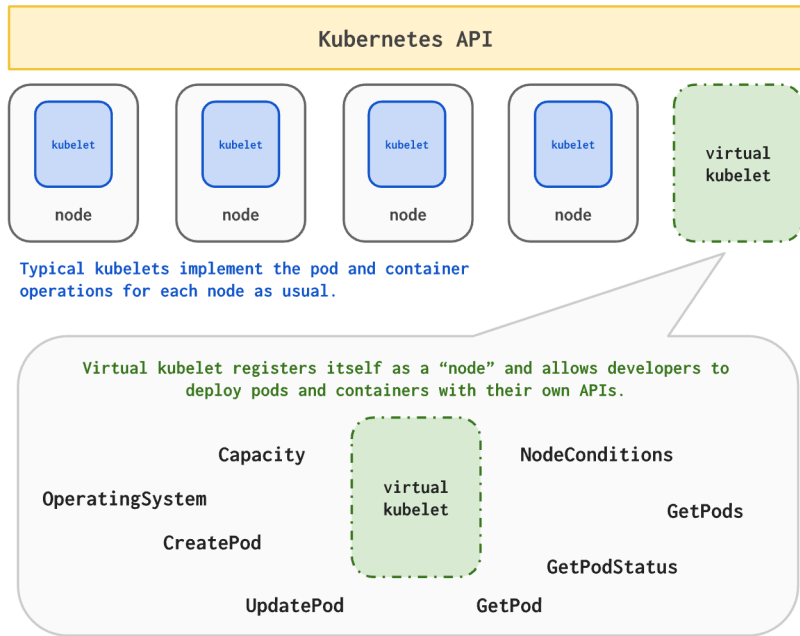
Virtual Kubelet

enabling the extension of the Kubernetes API into
serverless container platforms

Virtual Kubelet exposes federated or external compute capacity as Kubernetes nodes.

In Elastic Compute Pool, it bridges the base cluster with heterogeneous backend clusters and **large-scale** serverless GPU pools.

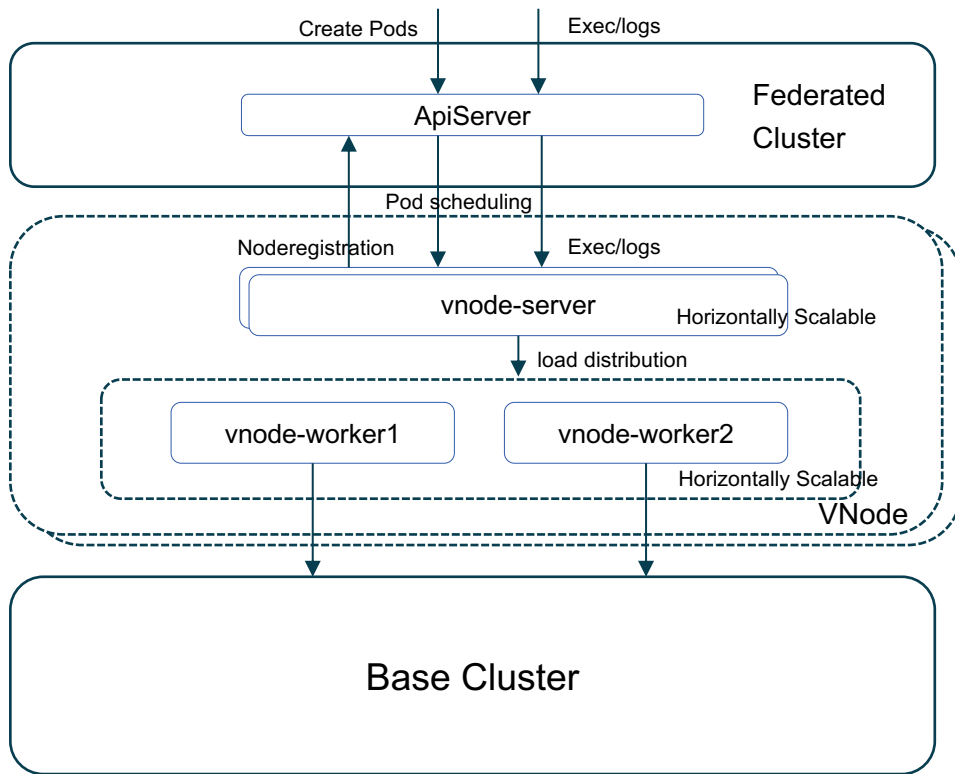
This keeps the user-facing API Kubernetes-native while enabling capacity federation, placement, and provisioning under the hood.



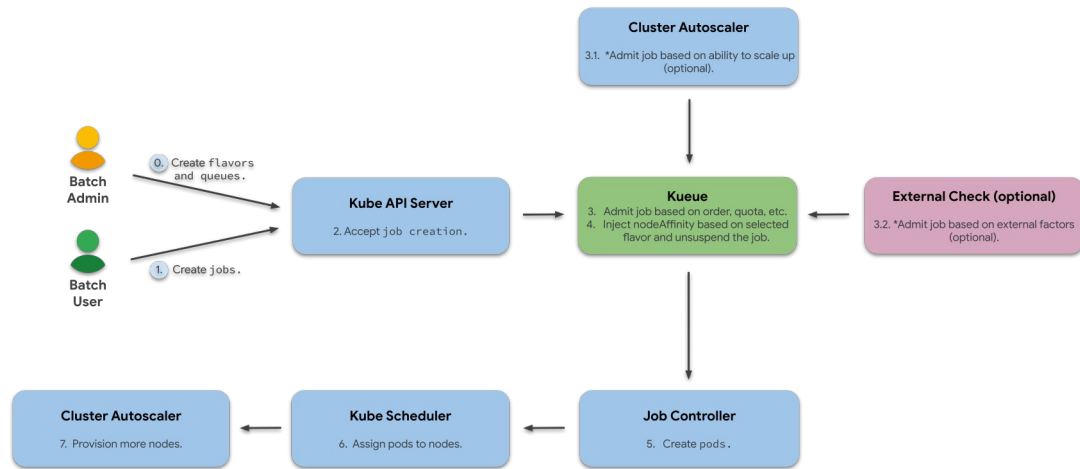
Large-Scale Federated Clusters

1. How can federated clusters manage resources at scale?
2. How can federated clusters stay highly available?

Horizontally scalable architecture



Capacity Allocation and Management - Kueue



Kueue

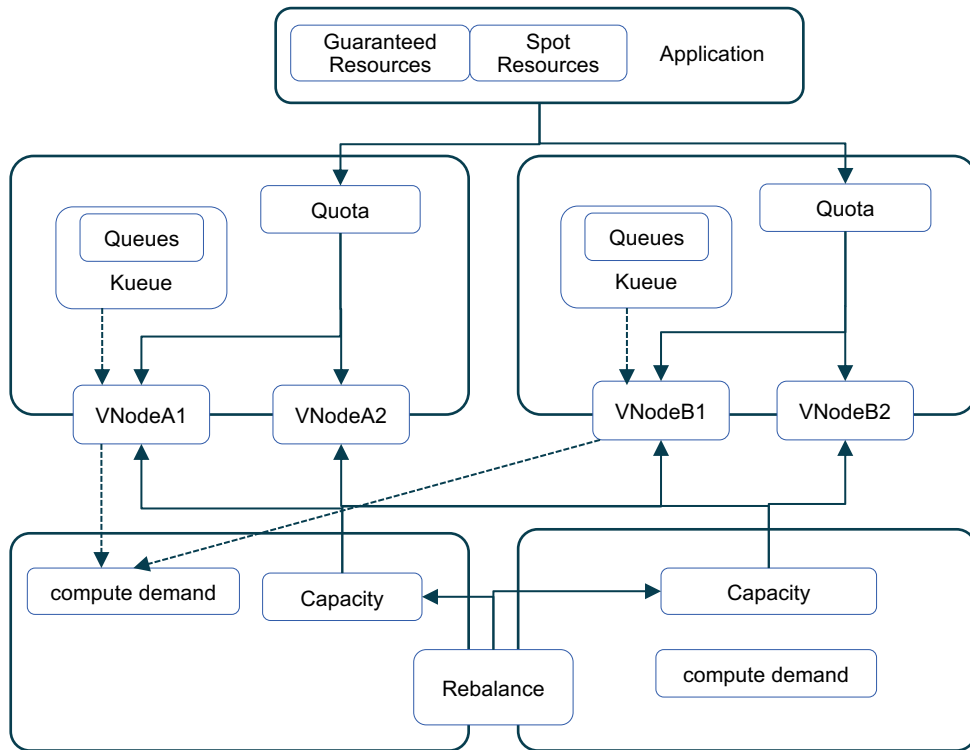
native **resource queuing** for standard
Kubernetes APIs

Kueue gives us a Kubernetes-native queuing and quota model.
We use it as the capacity control plane, then extend it with federated capacity and heterogeneous supply awareness.

Capacity Allocation and Management

Capacity and Quota allocation

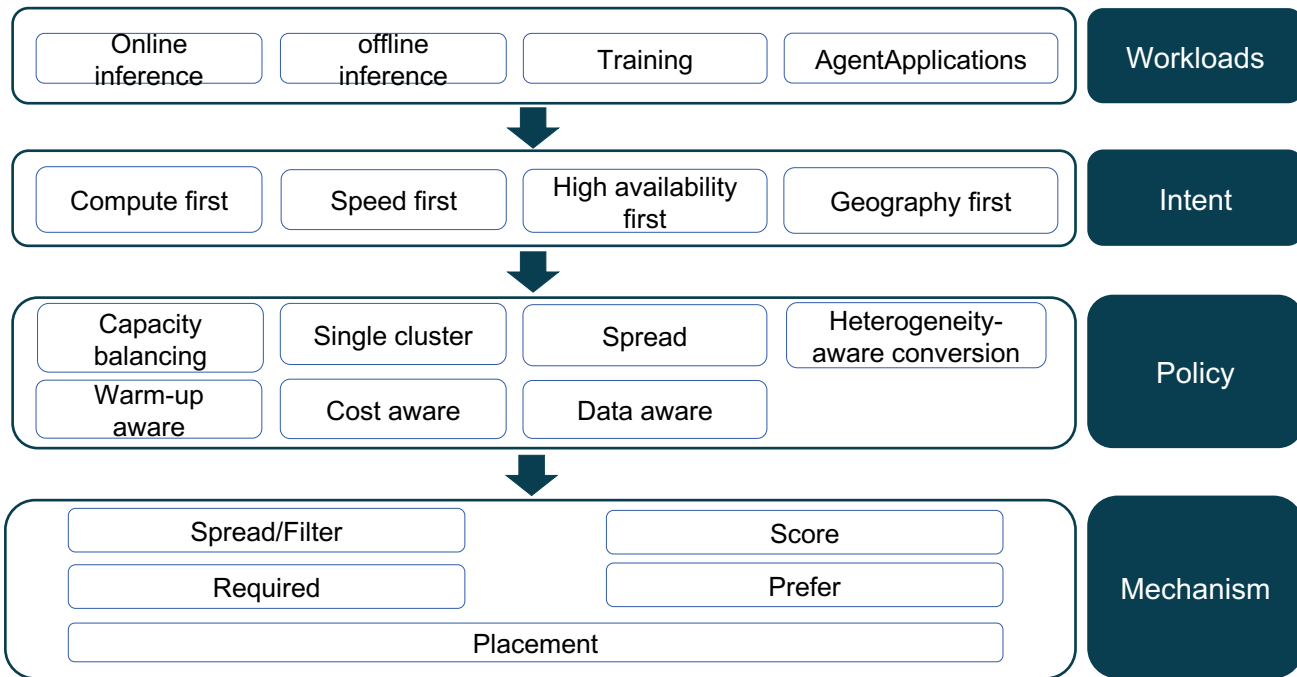
- Guaranteed Quota allocated by resource supply; inventory is balanced by demand.
- Best-effort capacity is dynamically partitioned by compute demand



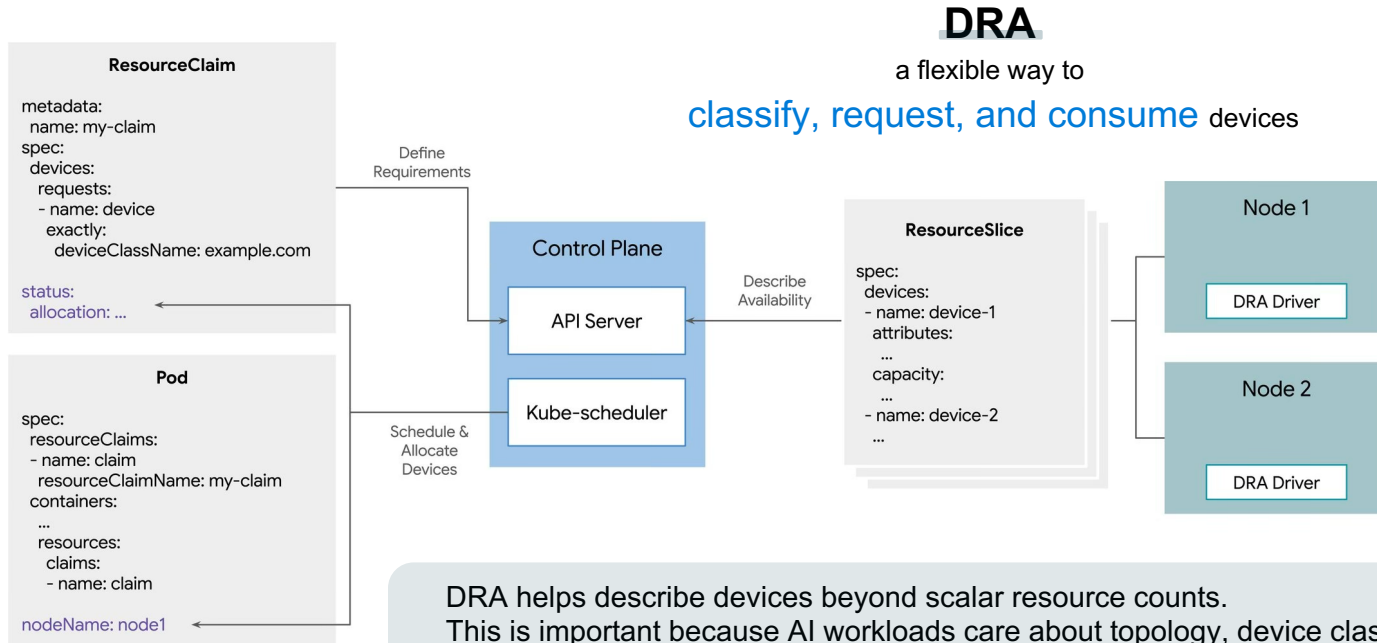
Scheduling Policies and Unified General / AI Scheduling

**Rich scheduling policies
support diverse compute
supply scenarios**

- Geography first
- Compute first
- Speed first
- Availability first

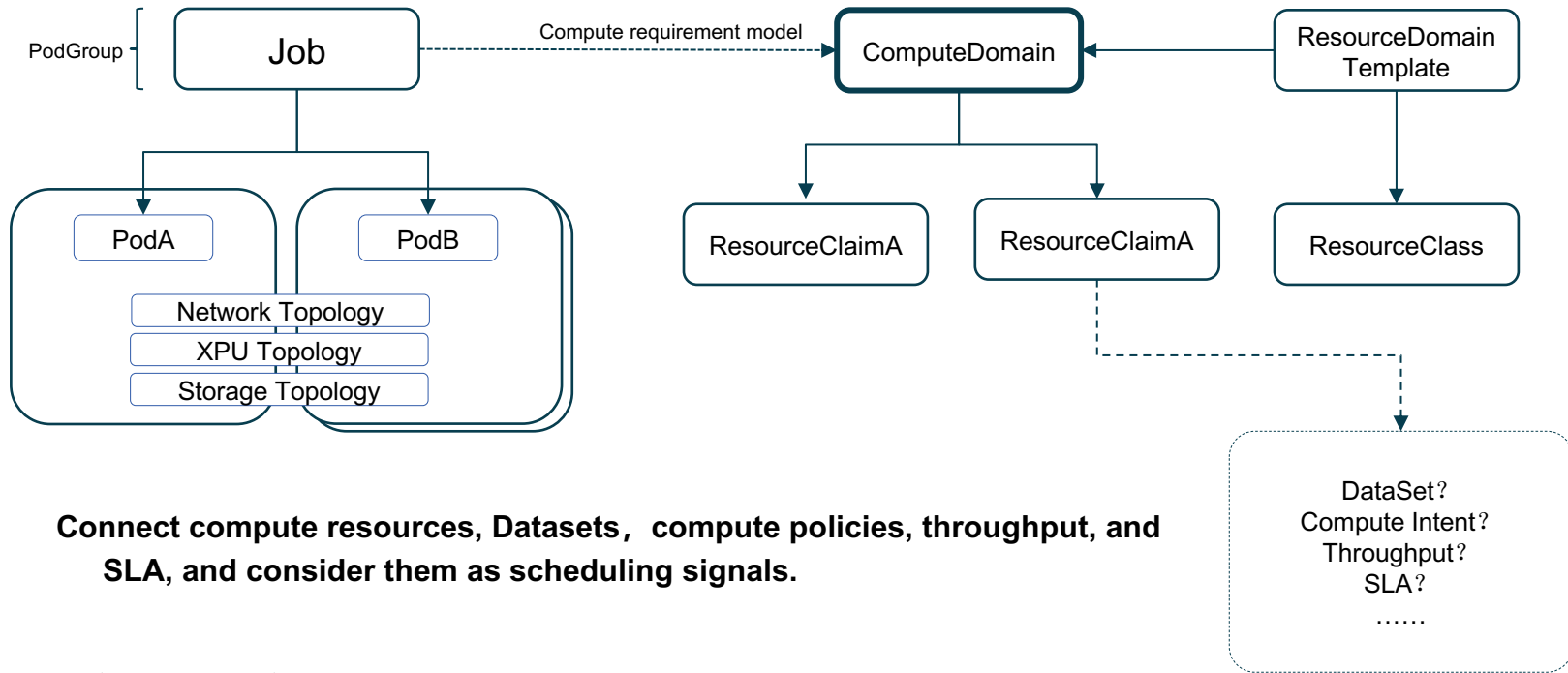


Compute Requirements and Descriptions - DRA



DRA helps describe devices beyond scalar resource counts. This is important because AI workloads care about topology, device class, and allocation semantics. We referenced these concepts to describe our compute abstractions.

From Device Requirements to Compute Intent



Connect compute resources, Datasets, compute policies, throughput, and SLA, and consider them as scheduling signals.

Data / Compute Co-Scheduling

Pre-training

Largest data demand:
PB/PB-scale
text/images/video; strongly
dependent on data locality

Post-training/RL

Medium-scale data:
preferences, trajectories,
and simulations; iteration
speed matters

AI Inference

Small per-request data;
large model weights;
optimize for low latency and
high throughput

Data processing

High data throughput;
CPU/IO/storage
coordination determines
upstream efficiency

Container images

Model weights

Datasets

Checkpoints, etc.

Data

Large data volume

Bring compute to data

Medium data volume

Predictive data movement

Small data volume

Move data to compute

GPU cluster

GPU cluster

GPU cluster

Compute Service

Lessons from Production & Future Roadmap



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Lessons from production

Experience & lessons:

1

GPU inventory is easy to count, but hard to make usable.

2

Data locality must be part of scheduling, not a post-scheduling operation.

3

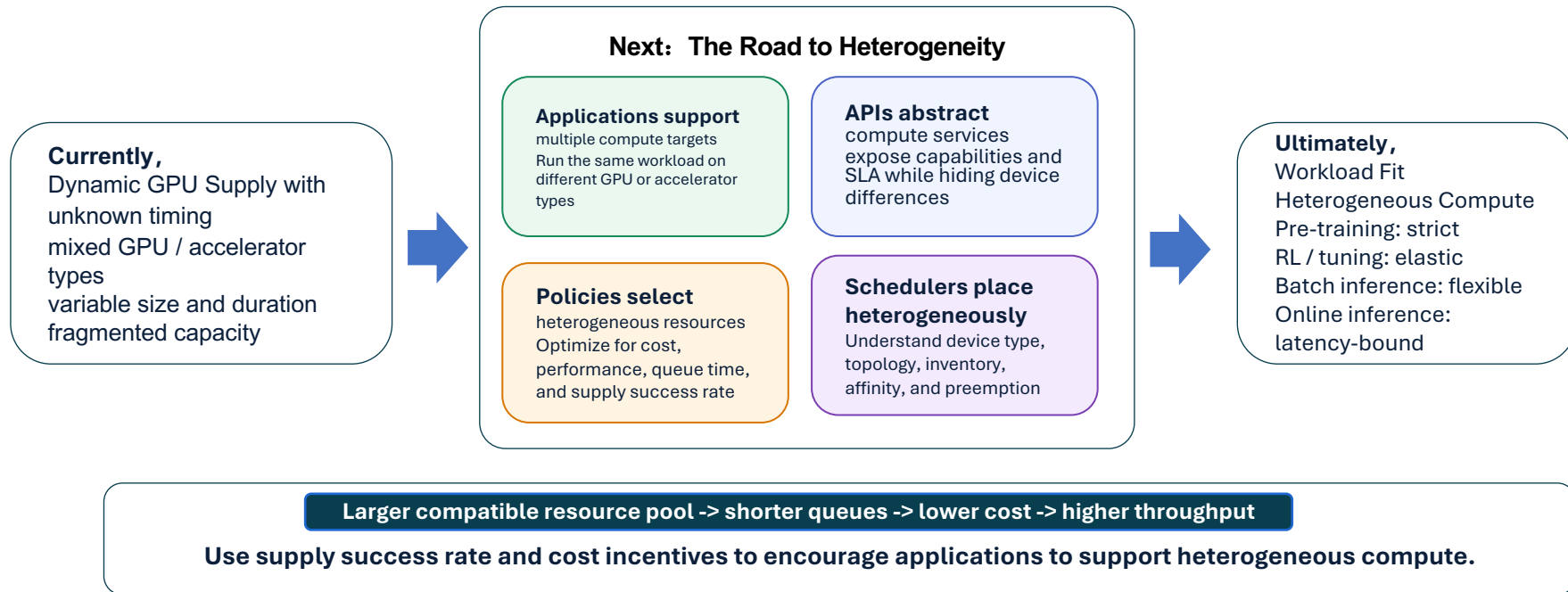
Queueing, quota, and placement are one system.

4

Heterogeneity only works when applications expose flexible intent.

Future Roadmap: From dynamic GPU supply to adaptive heterogeneous compute

Dynamic supply requires applications, APIs, policies, and schedulers to adapt together.



Thanks

Stop managing clusters.
Start managing compute capacity.





China 2026

